

Leveraging Model-Driven Architecture for Efficient Custom Instruction Utilization in Embedded Systems in C and Rust

Raphael Kunz^{*†}, Mayuri Bhadra^{*§}, Lijun Chen^{*†}, Stephanie Ecker^{*‡}, Wolfgang Ecker^{*†}

^{*}Infiniteon Technologies AG, Neubiberg, Germany

[†]Technical University of Munich, Munich, Germany

[‡]Chipglobe GmbH, Munich, Germany

{raphael.kunz, lijun.chen, ecker.external2, wolfgang.ecker}@infineon.com

Abstract—The push to support artificial intelligence in embedded systems introduces new challenges regarding software support of specialized hardware. The RISC-V instruction set architecture offers a promising platform for accelerators for said AI applications through custom instructions. Integrating these instructions into existing software codebases poses significant challenges, including manual adaptations and limited compiler support. This paper introduces a novel approach that generates hardware-optimized code utilizing custom instructions defined by a platform model. This method enhances the reusability of code and leverages design artifacts from the hardware design process to optimize the generated software in both C and Rust by generating intrinsic support for custom instructions through inline assembly. If the underlying platform does not allow the utilization of the specified custom instruction, an alternative implementation with optional global side effects is generated instead of the intrinsic. Thus, it expedites development by minimizing manual coding efforts and facilitates the seamless integration of hardware accelerators. An exemplary implementation of a matrix multiplication utilizing a custom multiply-accumulate instruction presented in this paper highlights the efficiency provided by this approach. Measurements based on this implementation show a reduction of the executed instructions by up to 75% while also achieving a 5-fold theoretical reduction in manual effort.

Index Terms—Model-Driven Engineering, Embedded software, Software engineering, Software architecture, Model-Driven Architecture

I. INTRODUCTION

The need for easily modifiable and reusable software-hardware interfaces has increased as embedded systems become more intricate and interconnected. Hardware accelerators for artificial intelligence for edge devices have especially shown a need for adaptable code. Integrating these accelerators into software requires knowledge about the system's architecture in such use cases. With the increasing popularity of the RISC-V Instruction Set Architecture (ISA), the potential for introducing custom instructions to interact with AI

acceleration has gained traction. The optimization potential of custom hardware via custom instructions contrasts with the additional effort required to integrate those accelerators into existing code bases. This need for software interfaces results in manual adaptations of existing code bases. Additionally, issues with compiler support for said custom instructions introduce further challenges and restrictions, exposing the developer to new potential memory safety violations.

One way to approach memory safety issues in embedded systems is to program in Rust. This young programming language is considered to help solve memory safety-related problems with its language features. With the built-in lifetime concept, Rust promises to solve memory leaks and invalid memory access. However, since Rust is a relatively new language and has not yet reached widespread adoption in the industrial community, a lack of platform support limits its usage. Conversely, this is not an issue for C code, which has widespread platform support but no built-in features targeting memory safety. Since at the moment support for both languages is necessary, our approach presents a way of generating for both Rust and C, to satisfy the demand for safe intrinsic function support, but also for support with legacy C code.

This paper presents a novel approach to generating code optimized for the provided hardware platforms by utilizing custom instructions. This is achieved using a model representing the platform on which the code gets executed. This model-driven technique allows for platform-specific optimizations tailored to the underlying platform.

The following gives a structure of the paper. The paper starts with a short list of related work in Section II, followed by a section explaining the background on the context of this paper in Section III. This is followed by a description of our methodology used to achieve the presented contribution in Section IV. Next, we present our results by highlighting the impact of our new approach with an example implementation of a matrix multiplication implementation utilizing a custom instruction via an intrinsic function in Section V. The paper closes with a conclusion in Section VI providing an outlook on future work.

This work was partially funded by the Federal Ministry of Research, Technology and Space (BMFTR) and ITEA within the GenerIoT project under contract number 01IS22084A, and by the BMFTR ZuSE (Scale4Edge) under project label 16ME0122K-1400, 16ME0465, 16ME0900, 16ME0901. § The author is at the time of publication no longer affiliated with either institution. She can now be reached via email at mayuri.bhadra12@gmail.com

979-8-3315-9379-7/25/\$31.00 ©2025 IEEE

II. RELATED WORK

The generation of embedded code for various systems has been explored extensively in previous studies, such as [1]–[3], with a significant portion of this research focusing on generating C code within the realm of embedded systems. In contrast to this trend, the 2024 Stack Overflow developer survey indicates a strong trend toward Rust as a widespread language for embedded systems [4]. To our knowledge, there is currently no existing Model-Driven code generation approach targeting embedded systems with Rust.

A compelling argument for transitioning to Rust code is its inherent memory safety feature. This feature contributes to developing safer and more secure code, as evidenced by Microsoft [5] and Google [6]. Their research found around 70% of vulnerabilities in their software to be memory-related. While the systems examined in these studies generally run client applications on general-purpose machines and therefore do not reflect errors related to embedded systems directly, they still indicate the inherent challenge of memory safety and its scale. Memory safety is particularly critical in embedded systems since incorrect access could lead to crashes or misbehavior. This risk is especially true for safety-critical embedded systems, which depend highly on reliable memory access. One noteworthy strategy for achieving memory safety without overhead is to port existing C code to Rust code. The research conducted by Fromherz and Protzenko [7] highlights an essential advancement as they present a subset of the C-language, which can be successfully transformed into safe Rust code.

While transforming existing C code into safe Rust code is commendable, sometimes Rust code has to be unsafe. This unsafe behavior is especially unavoidable when software directly interacts with hardware, as highlighted by [8]. This comprehensive analysis of 31,867 crates revealed that 7,507 contained unsafe code sections. Their research further analyzed the context requiring the unsafe keyword. One of the most prominent use cases was to interact with custom hardware. A pertinent example of such custom hardware extension can be an additional custom instruction added to an existing Instruction Set Architecture. An example of such a use case is the custom instruction introduced into the RISC-V ISA as presented in this work, which highlights the importance of a code generator being able to handle both safe and unsafe code.

The positive impact of utilizing such custom instructions via intrinsics in the context of embedded systems is well-recognized. However, properly implementing intrinsics in code can be a complex and potentially risky endeavor, as it involves the integration of assembly code. Changes in the underlying system architecture may necessitate a complete rewrite of the affected code sections. To address this challenge, [9] proposes a solution through a Model-Driven Architecture (MDA) based code generator that facilitates the generation of optimized C code tailored to the specific target architecture. The approach presented in [9] indicates a 70-fold decrease in manual effort.

It also aims to build on these findings by extending the proposed architecture to encompass Rust code generation in a way that isolates the unsafe sections, aiming to minimize the negative impact on Rust’s memory safety promise.

III. BACKGROUND

In the following three subsections, we present essential information to allow for a better understanding of the results presented in this paper. Section III-A provides a brief introduction to Model-Driven Architecture, followed by a Rust introduction in Section III-B. The final Section III-C will give a short overview of intrinsics, how they are modeled, and how one can utilize them.

A. MDA-Based 3-Stage Approach

Model-Driven Architecture is a software development approach that leverages the principles of Model-Driven Engineering. This methodology involves Metamodeling and automated transformations between models to facilitate the development of software systems [10]. The MDA framework consists of three primary models: the Computation Independent Model (CIM), the Platform Independent Model (PIM), and the Platform Specific Model (PSM). Additionally, the intermediate Platform Model (PM) is defined. The *Metagen framework*, as described in [11], adapts the 3-stage MDA methodology by incorporating elements from the revised MDA for generating embedded software code. This adaptation involves the use of three models:

- 1) The Model-of-Things (MoT), similar to the CIM, represents the formal requirements and specifications of the system under development, with explicit semantics.
- 2) The Model-of-Software (MoS), comparable to the PIM abstracts both target view and platform details, capturing the fundamental software structure in a platform-independent manner.
- 3) The Model-of-View (MoV), corresponding to the PSM, is the least abstract model, with a direct mapping to the target view, and is capable of generating valid code in the target view through unparsing.

All models are defined by Metamodels, for example, by Meta-Model-of-Things (MMoT), Meta-Model-of-Software (MMoS), and Meta-Model-of-View (MMoV). The transformation layers between the models are the Template-of-Software (ToS) and Template-of-View (ToV). The transformation process is facilitated by generator engines that leverage automatically generated APIs and transformation rules. These engines employ a Python-based Domain-Specific Language (DSL), designed to accommodate both hardware and software domains, enabling efficient and flexible transformations.

The extended MDA approach from [12], enhancing the flexibility in modeling as well as transformations for efficient and adaptable software code generation, is portrayed in Figure 1.

The MMoS includes Controlflow and ObjProps as two of the main Metamodels [10]. In these Metamodels the controlflow structures and the data types and their interpretations are defined respectively. In the controlflow structures,

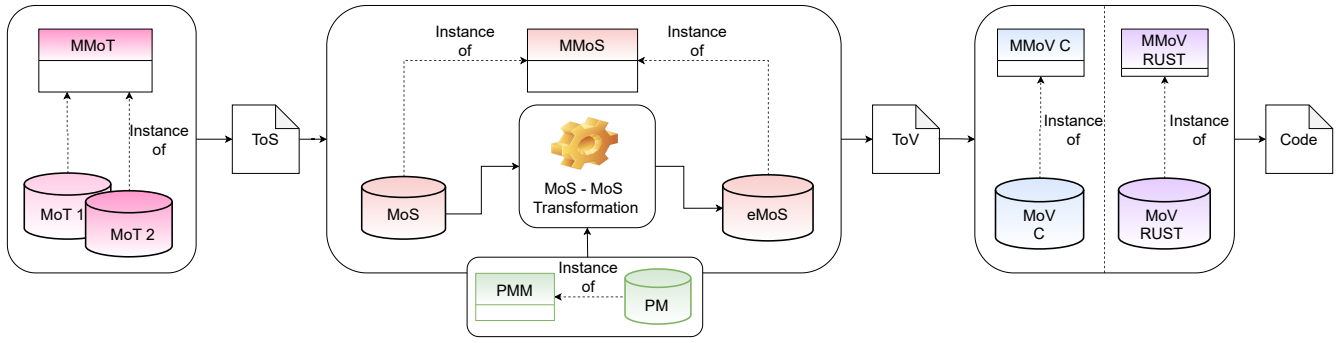


Fig. 1. Overview of the 3-layer MDA-based code generation approach, as presented in [9].

IntrinsicCall and InlineAssembly are defined, which enable the integration of user-defined instructions, assembly code and abstract intrinsics, which have a platform- and view-independent interface.

The ToS serves as a generator engine, leveraging the API frameworks derived from the Metamodel of Things (MMoT) and the Metamodel of Software (MMoS) to produce a view—and platform-independent MoS. The ToS is implemented using a Python-based DSL, which supports grammar-compliant, string-like expressions or a combination of both, depending on the application's complexity.

The elaboration process involves transforming the platform- and view-independent MoS into a platform- and view-dependent elaborated-Model-of-Software (eMoS) [10]. The eMoS, an instance of the MMoS, incorporates the most suitable implementations of abstract intrinsics for the target platform. To generate platform-specific code, the Platform-Model (PM) is crucial in transforming MoS to eMoS, providing technical specifications such as the instruction set architecture, memory hierarchy, and parallelism capabilities.

The View Language Description (VLD) [13] lies at the core of the framework for generating view-specific code, defining the syntax for the target view or its subset, and generating a Metamodel of View (MMoV). Subsequently, the ToV translates an eMoS into an MoV using the specific VLD.

During the MoS-eMoS transformation, abstract intrinsics are replaced with constructs specific to the target platform and view, thereby ensuring platform- and view-dependent code generation.

B. Rust

Rust is a multiparadigm programming language designed to produce high-performance code while allowing for a high level of abstraction [14]. Type safety is a fundamental design principle of the Rust language, which ensures that all calculations are performed on operands of the same data type. This forces developers to carefully consider the usage of correct data types, remember potential underflows or overflows, and handle them properly if necessary.

Rust's type safety extends to operands, which are not allowed to be pointer types, thereby guaranteeing the use of valid memory addresses. Another main characteristic of Rust

is memory safety, which is enforced by the use of a borrow checker. This compile-time analysis tool checks read and write access to variables and their lifetime. Each reference onto a memory section is restricted to only a singular write access at a time, while multiple concurrent read accesses are permitted.

Since Rust does not employ garbage collection, variables must be carefully managed throughout their lifetime. A value's lifetime begins when it is instantiated and ends when the scope of the definition is exited. It is essential to distinguish between a value and a variable, as the lifetime of a variable is not automatically tied to the lifetime of the associated value.

A possibility to bypass Rust's checks, which are intentionally restrictive, is to mark code as *unsafe* to enable certain operations, such as arithmetic operations on raw pointers, to be performed efficiently. However, this does not disable most type checks and borrow checks. Any violations of those checks will result in compile-time errors and must be resolved before compiling successfully. One of the language elements that must be marked as unsafe is inline assembly. According to the official documentation, this is due to the high complexity of analyzing assembly code and its associated risk [15].

C. Intrinsic Basics

In computer programming, intrinsics are functions or operations that directly map to specific instructions supported by a processor. These functions are typically exposed to the programmer through a high-level language, providing a way to access low-level hardware features without resorting to assembly language. Intrinsics enable developers to leverage the performance benefits of specialized processor instructions while writing code in a high-level language. One such feature is Single Instruction, Multiple Data (SIMD) instructions, which provide the developer with parallel processing capabilities. This approach efficiently utilizes the underlying hardware, improving performance in mathematical computations, multimedia processing, and other computationally intensive operations. In essence, intrinsics bridge the gap between high-level programming languages and low-level hardware, empowering developers to optimize their code for specific processor architectures while retaining the benefits of code maintainability and portability. This approach is particularly valuable in performance-critical applications where fine-tuning

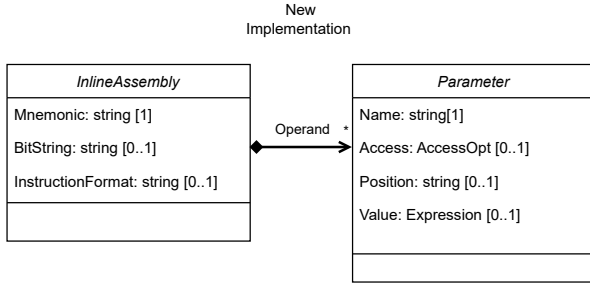


Fig. 2. MMoS support for modelling inline assembly code

code for hardware-specific features can significantly enhance overall system efficiency.

Typically, the compiler provides intrinsic support to a programmer. Bhadra et al. highlighted the possibility of generating intrinsic support from a model of a processor for C in [9]. This approach also utilizes the MDA approach and highlights how one can improve the generated code by using design artifacts from hardware generation to generate software support through intrinsics. However, the approach presented in this paper restricts itself to mnemonics, which are known to the compiler. This often excludes custom instructions since the compiler is unaware of them. To make a compiler aware of a custom instruction, one needs to modify the compiler for successful compilation. In their work, Bhadra et al. present a method for interacting with the MoS and the MoP during the elaboration stage, allowing for an automatic detection of necessary intrinsics. The approach presented utilizes an intrinsic library to look for supported intrinsics, which can be used in the ToS. The method also allows for the generation of alternative implementations in case the mnemonic is not supported by the system for which the code is generated.

IV. METHODOLOGY

In a previous paper [9], a method for generating and elaborating intrinsic functions based on the hardware model of a processor in C was demonstrated. In this paper, the authors take a MoS and do a single pass over the tree, identifying calls to an intrinsic function and inserting the corresponding code support into the MoS. Our approach builds on these findings by extending the intrinsic library to support side effects, adding recursive elaboration to ensure the resolution of intrinsics, which require intrinsics themselves, modifying the way inline assembly code is generated in C, while also introducing Rust as a new view, thereby highlighting the view independence of our abstract MoS. Our approach is designed to eliminate the need for custom compilers, ensuring vendor independence in the toolchain when compiling for the target. In the following two sections, the contributions of this paper are presented in more detail.

A. Cross Language Support for custom instructions

The major contribution of this paper is to highlight a method for generating code that utilizes new custom instructions with-

out requiring the development of a new compiler. To achieve this, we decided to use LLVM's "insn" language assembler directive [16]. This decision limits our generated code support to RISC-V-based CPUs, as the "insn" assembler directive is target-specific to RISC-V. From a language perspective, our selection has a limited negative impact, since LLVM has frontend support for C and Rust. In the case of Rust, the rustc compiler is built as a frontend on top of the LLVM backend [17], while C has Clang as a frontend [18]. There are two kinds of support for this directive: format- or length-based. The length-based approach requires the code generator to encode the complete instruction, including the registers, directly in bits. In contrast, the format-based approach allows the generator to specify the RISC-V ISA format that the custom instruction adheres to. This, in turn, limits a custom instruction to adhere to one of the RISC-V instruction formats.

Generating inline assembler code based on this feature from the presented PM required modifications to the MMoS and the MMoV of both Rust and C. Our MMoS of the inline assembly elements can be seen in Figure 2. One can see that this model contains two optional attributes, which are used in our elaboration. The first optional attribute is the InstructionFormat, which stores the information about the format to which this instruction adheres. The previous implementation was solely based on the presence of the mnemonic.

The second optional attribute is called BitString. This is an abstract string responsible for storing information about the expected opcode. During the elaboration, the data stored in the PM, as shown in [9], is transformed from a tree of modelling options into a concrete dictionary of mnemonics mapped onto partially defined bit strings. They are partially defined, since the values of the associated parameters are not hard-coded in most cases. In this paper, this bit string will be called the fingerprint of the instruction, since this information should be unique to an instruction. It can later be used to identify the closest fitting instruction format.

Since an instruction consists of an opcode and parameters, the information on the parameters used by this instruction is modelled in a separate class. This class contains information on the parameter, including its access type, name, optional value, and optional position. This position is designed to model the information on the bits this parameter uses in the instruction.

In addition to the changes to the parameters in the MMoS, the parameter view in the PM was extended by adding a link configuration. Since some intrinsics might require more control over an instruction's parameters, the PM provides the option to force specific configurations. This also allows the control of return values and types of instructions, which might be required in some cases, where non-standard programming types, like custom structs, are used. The extended control also lets the developer specify intrinsics that directly interact with special registers. Firmware developers sometimes require this level of control to enable interaction with special status registers. This highlights that the usage of our approach is not limited to compiler support but provides a way for intrinsic

Listing 1. Example for an `.insn` assembly directive with an R-type format for a MAC instruction in Rust with opcode = 0xb, funct3 = 0x4, funct7 = 0x0 and MAC-thread 8

```
pub unsafe fn _intrinsic_impl_mac_4_1(
    val_a: i32, val_b: i32) -> () {
    use core::arch::asm;
    asm!(
        ".insn r 0xb, 0x4, 0x0, x8,
        {rs1_addr_32}, {rs2_addr_32}",
        rs1_addr_32 = in(reg) val_a,
        rs2_addr_32 = in(reg) val_b
    );
}
```

firmware support.

During our elaboration, this metamodel is used to create inline assembly support as proposed in [9] based on the PM and hardware model information. Most importantly, at this level, the decision on whether to use the “`.insn`” style or the mnemonic style has not yet been made, but instead will be made during the ToV, since this transformation is language and therefore view-specific.

During the ToV, if an inline assembly element is encountered, a shared function transforms this model to a formatted string. This function analyses the content of the model. The format information will be set if the instruction was specified in the platform model to be part of an active and standard RISC-V instruction. Therefore, if this information is set, the mnemonic can be generated directly.

If the instruction’s format information is not standard or not active for the target, the fingerprint gets used. Based on the data stored in the fingerprint, a factory-style transformer tries to match the bit pattern with the pattern information required and specified for the standard formats [19]. In addition, the location and information of the parameters are used to calculate this match. This information is also used to generate the correct order in the inline assembly call and generate the correct linkage between the inline assembly code and the function wrapper. If a fitting format is found, this “`.insn`” style string is generated for the model and inserted into the code.

By splitting the generation of the inline assembly support into those two stages, we can create a clear separation of concerns. While our MMoS model and elaboration are responsible for identifying suitable implementations, our ToV focuses on how those implementations can then be provided view-specific, allowing for easy processing by the compiler. An example of one such generated intrinsic for a custom instruction can be found in Listing 1.

While the first part of this section highlights how the inline assembly support is getting generated, the challenge of supporting the case where the compiler or the underlying system does not support a required intrinsic function needs some highlighting. This challenge is solved by generating an alternative implementation and is also part of the work done

by Bhadra et al. [9]. In this work, a way of modelling an alternative implementation is presented. While this model is sufficient for most instructions, we identified some limitations that our proposed methodology addresses.

The first limitation we identified is the requirement of modelling side effects for more complex instructions. An example of an instruction requiring or dealing with side effects might be a cryptographic function, like a hash, where, after each execution, an internal state is updated. To solve this limitation, we propose an addition to the alternative implementation, which allows for the introduction of a side effect that can be generated and handled as a global variable. This gives the generator the freedom to explicitly model and share a state between alternative implementations of custom instructions in case they are unavailable.

The second limitation we identified is the limited analysis of the equivalent implementation. Our research shows that this equivalent implementation can itself contain an intrinsic. Therefore, the equivalent implementation can’t simply be inserted during elaboration, but must be elaborated on. This requires the elaboration to be performed recursively on the to-be-inserted intrinsic functions.

B. Rust-Code Generation for Intrinsics

An additional contribution achieved by this work is the generation of intrinsics in Rust. Previous work focused on generating intrinsics specifically for C. To achieve Rust support, the inline assembly feature from Rust is utilized [15]. Since the inline assembly feature is marked as unsafe, the decision was made to encapsulate this code in an unsafe block and not expose this unsafe interface through the function. This makes sense since the alternative section might contain safe Rust code. This approach minimizes the exposure of unsafe code to the rest of the code, increasing the benefit of Rust’s safety concept.

For the final unparsing of the inline assembler instruction and the surrounding function, the existing MMoV was enhanced to support this assembly structure, similar to the modifications done to achieve unparsing support for C in [9]. Our modifications introduce a new inline assembler element in the VLD file, abstracting the required syntax in an EBNF notation while inserting the information stored in the MoS, shown in Figure 2, directly into the generated code. To achieve the correct generation of Rust code, a modification to the ToV was necessary, ensuring the proper translation of the eMoS into the MoV. This was achieved by adding the Python class of the Inline Assembly MMoS to the existing dispatcher for transformation.

V. RESULTS

In this section, we present the results of our experiments, highlighting the improvements achieved by the presented work. To highlight the diverse applicability of our approach, we demonstrate the generation of an intrinsic for a packed MAC instruction. MAC instructions can be utilized in a wide range of applications, including AI applications and

Listing 2. alternative implementation for MAC instruction in Rust

```
static mut mt_val: i32 = 0;

pub fn unsafe _intrinsic_impl_mac_4_1(
    val_a: i32, val_b: i32) -> () {
    let mut r_a: i32 = val_a;
    let mut r_b: i32 = val_b;
    mt_val = 0;
    let mut i: u32 = 0;
    while i < 4 {
        mt_val += (r_a&255) * (r_b&255);
        r_a = r_a >> 8;
        r_b = r_b >> 8;
        i = i + 1;
    }
}
```

cryptographic functions. For this, we designed a ToS modeling a matrix multiplication calculation of two matrices with a common vector length of a multiple of 4. We utilize the multiply-accumulate operation (MAC) presented and modelled in [20] to perform the matrix multiplication. The instruction is designed not only to calculate a scalar-MAC but also to implement a packed-MAC, which accumulates the results of the multiplications in a thread register. In this case, a thread refers to a special register that can not be accessed directly by the processor, but only by the hardware accelerators. The packed-MAC is implemented in a way that allows for four simultaneous MAC executions. This implementation of the operation is also integrated into our RISC-V MoT.

The interface of the MACR instruction follows an R-Type instruction. Since the operations require read and write access to the thread register, no direct interaction with the register is possible. Therefore, the destination register is not used directly, but rather as an index to an array of threads, indicating which thread should be used. This results in a more complex call since the destination register must be manually set and hard-coded in the assembler code, especially since the packed instruction is only performed when a specific thread is used. To handle the behavior of the thread register in an alternative implementation, a global variable storing the intermediate values is generated alongside the logic. In the case of the alternative implementation, the number stored in the MAC thread is stored in a global variable. This allows for the accumulation without changing how the intrinsic would be called or handled. The alternative implementation in pure software is shown in Listing 2.

Based on this instruction, we can generate three distinct implementations for C and Rust, each resulting in six versions of this matrix multiplication. The first implementation is purely software-based, the second utilizes the scalar setting of the MAC instruction, and the third utilizes the packed setting of the MAC instruction. To validate this generated code, we

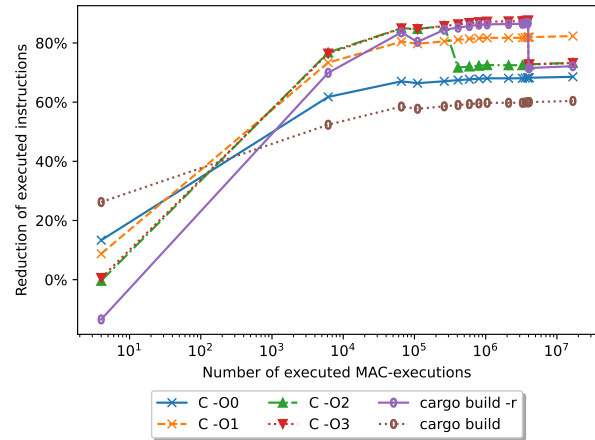


Fig. 3. Reduction in executed instructions for the packed MAC-instruction compared to executed instructions of the pure software implementation

compile all of them for a 32-bit base integer RISC-V with a standard multiplication and division extension (M-extension). The C code is compiled with Clang, while cargo build is used for Rust code. As test stimuli, two matrices with randomly selected values are used. We generated 17 different matrix size combinations to highlight consistency in performance improvements. The size combinations are currently limited to cases where the common dimension of both matrices needs to be a multiple of 4. Each of those scenarios was compiled in case of C code with -O0, -O1, -O2, and -O3, while for Rust code “cargo” and “cargo build –release” were used. The resulting binary files are then simulated on a modified whisper-simulator [21]. This resulted in a total of 306 simulations.

To present the results of these runs, we calculated the ratio of executed instructions between the pure software implementation and the implementation utilizing an intrinsic. Based on the MAC instruction, a reduction of around 50% was expected in the scalar-MAC case, since this combines a multiplication and an addition. An additional 75% reduction in executed instructions for the packed-MAC is expected, as four MAC calls are grouped into one. This results in a total expected possible reduction in executed instructions by 87.5% when comparing the packed-MAC results with a pure software calculation. The resulting reductions can be seen for the packed-MAC in Figure 3 and for the scalar-MAC in Figure 4.

Examining the results of the scalar-MAC, as shown in Figure 4, reveals that the performance remains stable but does not achieve the expected 50% reduction. This is most likely due to the additional code for iteration having a heavier impact than expected. The results of the packed-MAC achieve the expected reduction of 87.5%. However, this reduction is not reliably achieved in all our tests. Around 3,993,600 MAC instructions, one can observe a drop from an 87.5% decrease to a 72.7% reduction in executed instructions. Remarkably, this drop is not only happening in Clang but also in Cargo.

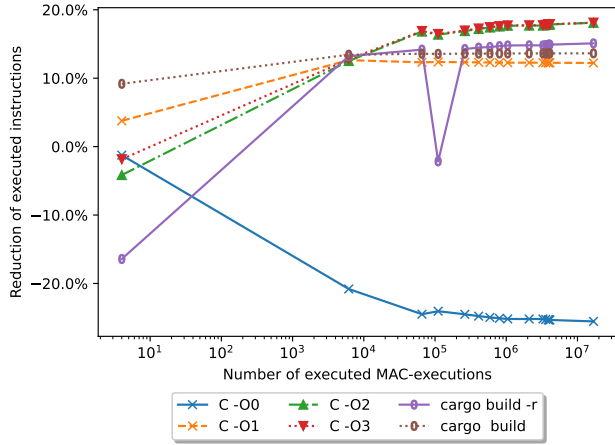


Fig. 4. Reduction in executed instructions for the scalar MAC-instruction compared to the executed instructions of the pure software implementation

An analysis of the intermediate representation from the Clang build revealed that -O3 utilizes loop unrolling aggressively. This stops around 3,993,600 MAC instructions, at which point LLVM falls back on function calls without loop unrolling. Without this, the ratio stabilizes around 73%, which aligns with our observations from the scalar MAC, indicating the code required for iterations dominates over the possible savings from simple MAC instructions.

Since our paper presents implementation in C and Rust, we also want to highlight the ratio of executed instructions between C and Rust, as shown in Figure 5. For this, we divided the number of executed instructions for the Rust binaries by the number of executed instructions for the C binaries. For our calculations, we selected C -O0 optimization to compare it with the cargo debug build, and -O3 optimization to compare it with the cargo release target. One can see that Rust starts with a lower overhead, but quickly settles at a stable ratio between 0.9 for the optimized full software builds and 2.6 for the debug build using the packed-MAC instruction. These ratios are to be expected and in line with what Zhang et al. presented in [22].

To further highlight not only the performance advantage of code generation in executed instructions, but also the reduction in manual development effort, an overview of the number of generated lines of code is given in Table I. Our approach generates 662 source lines of code (SLoC) while requiring only 128 SLoC for a ToS. This results in a 5-fold decrease in manual effort. The context of this reduction is significant, as the generated code interacts with different types of hardware accelerators or their absence. Previous work presents a high reduction in manual effort, achieved by this generation approach, in the range of 70 to 126-fold, as reported in [9] and [12]. While one could argue that those numbers are achievable through copy-pasting code, one might overlook the risks associated with such development methods, like forgetting to change certain checks or types. The stark contrast

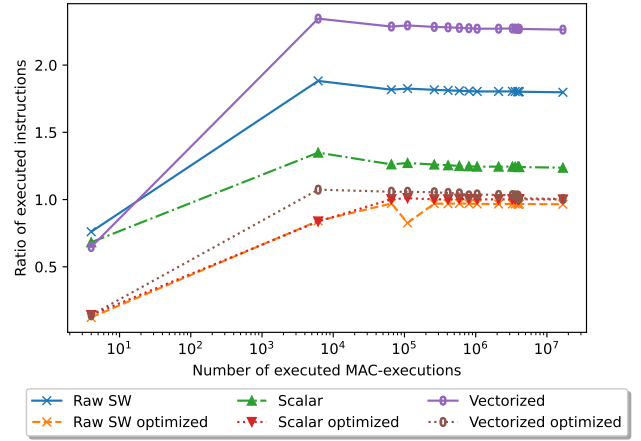


Fig. 5. Ratio between executed instructions for C code against the executed instructions for Rust code. A ratio of X means Rust requires X times the amount of executed instructions compared to the generated C code in the same measurement setup.

TABLE I
OVERVIEW OF CODE SIZES AND THEIR FREQUENCY

Source Files	total SLoC	Effort frequency
ToS	128	once
Intrinsic Library	1,358	once per target platform
C code	385	per target & per variant
Rust code	277	per target & per variant

in factors may be related to the ToS, as the one presented in this work is closely tied to the intrinsic function and does not allow for a broad range of possible generation targets compared to those highlighted in [9], [12]. Nonetheless, the improvement is interesting, especially in embedded systems, since assembler code typically requires more manual effort than average application code due to its inherent complexity.

VI. CONCLUSION

This paper presents a code generation approach for C and Rust that generates target-optimized code utilizing intrinsics for custom instructions in both languages. Our work introduces a new level of optimization in the context of code generation, enabling the generation of hardware-optimized code in Rust and C by utilizing custom ISA extensions. Additionally, we present additions that enable modeling side effects in alternative implementations in cases where intrinsic support is missing. By providing this code in both Rust and C, we encourage the selection of the best-fitting language, allowing for a trade-off between the safety promises made by Rust and the additional performance improvements provided by C, as highlighted by our results. This approach allows for a theoretical 5-fold reduction in development effort compared to manual effort, while achieving code performance comparable to current state-of-the-art implementations. The results in this paper enable us to explore further the generation of more complex software structures, such as firmware and drivers,

where intrinsic functions play a crucial role in interacting with the hardware. This also opens the possibility of verifying the hardware-software co-design by utilizing approaches like those highlighted by DeepSpec [23].

Our approach is limited to custom ISA extensions, as well as all existing instructions that follow the existing RISC-V ISA formats. Our approach is also limited to only being compilable with an LLVM-based compiler. These limitations indicate that further research is necessary to enhance the automatic utilization of custom instructions in code generation, enabling the generation of safe code and the automatic creation of the intrinsic library from the platform model, thereby reducing manual development effort even further. Opening up the “`insn`” instruction to not only be usable for RISC-V targets would allow our approach to be utilized for x86 instructions as well, by introducing an alternative set of abstract factories for fingerprint matching.

REFERENCES

- [1] A. Suljkanović, D. Majstorović, G. Milosavljević, and I. Dejanović, “MDA approach in designing real-time embedded systems,” in *2016 Zooming Innovation in Consumer Electronics International Conference (ZINC)*. IEEE, 2016, pp. 4–7.
- [2] M. Fredj, A. Radermacher, S. Gerard, and F. Terrier, “eC3M: Optimized model-based code generation for embedded distributed software systems,” in *2010 10th Annual International Conference on New Technologies of Distributed Systems (NOTERE)*. IEEE, 2010, pp. 279–284.
- [3] E. Borde, P. H. Feiler, G. Haïk, and L. Pautet, “Model driven code generation for critical and adaptive embedded systems,” *SIGBED Rev.*, vol. 6, no. 3, Oct. 2009. [Online]. Available: <https://doi.org/10.1145/1851340.1851352>
- [4] S. Overflow. (2024) Stack Overflow Survey 2024. [Online]. Available: <https://survey.stackoverflow.co/2024/>
- [5] M. S. R. Center. (2019, July) A Proactive Approach to More Secure Code. Accessed: 2025-01-19. [Online]. Available: <https://msrc.microsoft.com/blog/2019/07/a-proactive-approach-to-more-secure-code/>
- [6] C. Project. (2020) Memory Safety. Accessed: 2025-01-19. [Online]. Available: <https://www.chromium.org/Home/chromium-security/memory-safety/>
- [7] A. Fromherz and J. Protzenko, “Compiling C to Safe Rust, Formalized,” 2024. [Online]. Available: <https://arxiv.org/abs/2412.15042>
- [8] V. Astrauskas, C. Matheja, F. Poli, P. Müller, and A. J. Summers, “How do programmers use unsafe Rust?” *Proceedings of the ACM on Programming Languages*, vol. 4, no. OOPSLA, pp. 1–27, 2020.
- [9] M. Bhadra, S. Ecker, D. Albert, R. Ramaiah, S. Prebeck, and W. Ecker, “Automated Intrinsic Support for ISA Extensions: Enhancing Software Generation for RISC-V and Beyond,” in *2024 IEEE Nordic Circuits and Systems Conference (NorCAS)*, 2024.
- [10] M. Bhadra, D. Lopera, R. Kunzelmann, and W. Ecker, “A model-driven architecture approach to accelerate software code generation,” in *Proceedings - 2024 7th International Conference on Software and System Engineering, ICoSSE 2024*. Institute of Electrical and Electronics Engineers Inc., 2024, pp. 23–30.
- [11] W. Ecker and J. Schreiner, “Introducing Model-of-Things (MoT) and Model-of-Design (MoD) for simpler and more efficient hardware generators,” in *2016 IFIP/IEEE International Conference on Very Large Scale Integration (VLSI-SoC)*, 2016, pp. 1–6.
- [12] M. Bhadra, D. Albert, U. Yun, R. Kunzelmann, D. Sanchez Lopera, and W. Ecker, “A Model-Driven Architecture Approach to Efficient and Adaptable Software Code Generation,” in *MBMV 2024; 27. Workshop*, 2024, pp. 196–203.
- [13] J. Schreiner, F. Willgerodt, and W. Ecker, “A new approach for generating view generators,” in *Design and Verification Conference-US*, 2017.
- [14] Rust Development Team. (2025) The Rust Language Reference. Accessed: 2025-02-06. [Online]. Available: <https://doc.rust-lang.org/reference/>
- [15] (2024) The Rust Reference: Inline Assembly. Accessed: 2025-01-19. [Online]. Available: <https://doc.rust-lang.org/reference/inline-assembly.html>
- [16] The LLVM Project. (2025) RISC-V LLVM Documentation. Accessed: 2025-02-06. [Online]. Available: <https://llvm.org/docs/RISCVUsage.html>
- [17] The Rust Project Developers. (2025) Rust Compiler Developer Guide. Accessed: 2025-02-06. [Online]. Available: <https://rustc-dev-guide.rust-lang.org/overview.html>
- [18] The LLVM Project. Clang - A C language family frontend for LLVM. Accessed: 2025-02-06. [Online]. Available: <https://clang.llvm.org/>
- [19] A. Waterman, Y. Lee, D. A. Patterson, and K. Asanovic, “The RISC-V instruction set manual, volume I: User-level ISA, version 2.0,” *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-54*, p. 4, 2014.
- [20] S. S. Prebeck, “HW-Acceleration for Edge-AI,” Ph.D. dissertation, Technische Universität München, 2024. [Online]. Available: <https://mediatum.ub.tum.de/1705588>
- [21] Chips Alliance. (2025) RISC-V instruction set simulator (ISS). Accessed: 2025-02-06. [Online]. Available: <https://github.com/chipsalliance/VeeR-ISS>
- [22] Y. Zhang, Y. Zhang, G. Portokalidis, and J. Xu, “Towards understanding the runtime performance of rust,” in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–6.
- [23] A. Appel, A. Chlipala, B. Pierce, Z. Shao, S. Weirich, S. Zdancewic, and L. Beringer, “The Science of Deep Specification,” <https://deepspec.org/>, accessed: 2025-07-23.